

# CS598 HPC. Homework - Parallel LU Factorization

Due: Next week.

## Problem Statement:

In this exercise we will explore several aspects of LU factorization with a distributed-memory/message-passing model. Specifically, we are interested in solving  $A\mathbf{x} = \mathbf{b}$ , with  $A$  a general, dense, distributed  $m \times m$  matrix. To avoid writing separate triangular solves, we will factor the augmented system,  $\tilde{A} := [A \mid \mathbf{b}]$  and therefore develop a scheme to factor an  $m \times n$  system (e.g., with  $n = m + 1$  in this particular circumstance).

We will consider two principal variations:

- development/implementation of partial pivoting, and
- development/implementation of BLAS3-based block-factorizations.

## Recommended Reading:

- van de Velde, Chapter 4

## Simple Serial Algorithm:

The snippet below illustrates the basic (BLAS1) algorithm for serial LU factorization of a matrix that does not require pivoting (e.g., the diagonally-dominant or SPD case).

```
%    LU Algorithm 1
1    mn = min(m,n);
2    for k=1:mn ;
3        pivot_inverse = 1./A(k,k);
4        pivot_row      = A(k,k+1:end);
5        mult_col        = pivot_inverse*A(k+1:end,k);
6        A(k+1:end,k+1:end) = A(k+1:end,k+1:end) - mult_col*pivot_row' ; % rank-one update
7        A(k+1:end,k)      = mult_col; % save lower-triangular factor
8    end;
```

Note that the update step in Line 6 is an outer product of the form

$$A_{\tilde{I},\tilde{I}} = A_{\tilde{I},\tilde{I}} - \mathbf{c}_k \mathbf{r}_k^T, \quad (1)$$

where  $\tilde{I} = [k+1 : \text{end}]$  is the appropriate range for the trailing submatrix in the factorization step. Note also that if we replace Lines 4 and 6 with

```
4        pivot_row      = A(k,k:end);
6        A(k+1:end,k:end) = A(k+1:end,k:end) - mult_col*pivot_row'
```

then we will produce zeros in column  $k$  for  $i > k$ , which is the intent of Gaussian elimination (i.e., LU factorization), namely, to convert  $A\mathbf{x} = \mathbf{b}$  into an upper-triangular system that yields the same answer.

**Quick Quiz** for LU Algorithm 1. Assume  $m = n$ .

- How many divisions?
- Which line has the most work?
- How much work associated with that line, for the *full factorization*?
- How many memory refs associated with that line, for the *full factorization*?
- Assume you can get 30 GFLOPS per rank and perfect  $P$ -fold speed-up.  
How large a system could you factor in 1 minute using  $P=32$  processors?

As we get into the parallel algorithm we will need to fully understand the data interactions implied by the outer-product update, Step 6. To this end, we expand that step in LU Algorithm 1 to yield the following.

```
%      LU Algorithm 1b
1      mn = min(m,n);
2      for k=1:mn ;
3          pivot_inverse = 1./A(k,k);
4          pivot_row      = A(k,k+1:end);
5          mult_col       = pivot_inverse*A(k+1:end,k);
6.1      for j=k+1:end
6.2      for i=k+1:end
6.3          A(i,j) = A(i,j) - mult_col(j)*mult_col(i)
6.4      end;
6.5      end;
7      A(k+1:end,k)      = mult_col;          % save lower-triangular factor
8      end;
```

A couple of preliminary comments about this expansion. First, 6.1 and 6.2 are indented equally because they are peer loops—they are simply advancing the indices  $i$  and  $j$  and are interchangeable. If there was a variable update between 6.1 and 6.2 (or between 6.4 and 6.5), I would have indented 6.2–6.4. Second, they are carefully nested, with the  $i$  loop running fastest because we are assuming a *column-major* ordering of  $A$ , which implies that  $a_{11}, a_{21}, a_{31}, \dots$ , are adjacent in memory (i.e., are separated by *unit-stride*.) Because CPUs pull in a full cache line (typically four  $\times$  64-bit words) when a variable is accessed, one can expect that  $[a_{11}, a_{21}, a_{31}, a_{41}]$  will all be retrieved from (slow) main memory to (fast) L1 cache when  $a_{11}$  is accessed, assuming that  $a_{11}$  is at the start of a cache line. With the  $i$ -loop, 6.2–6.4, *inside* the  $j$ -loop, we are accessing  $a_{i,j}$  variables in order,  $i = k+1, \dots, m$ , and thus immediately using all 4 cached variables. If instead, we were working with the  $j$ -loop on the inside, we would be working with just a single cached variable per column and the performance would be significantly worse (e.g.,  $\approx 4\times$  slower). Note that matlab and Fortran use column-major ordering. C really does not work well with multidimensional arrays and high-performance codes therefore use one-dimensional arrays with their own indexing code. Therefore one can (and should) use column-major ordering for high-performance linear algebra code in C. (Why? - Because one can then also call high-performance LAPack Fortran code from C.)

Now, the main point about the expansion 6.3 is that we can see that the update of  $a_{ij}$  is

$$a_{ij} = a_{ij} - c_i \cdot r_j. \quad (2)$$

That is, updating  $a_{ij}$  requires access to itself, to  $c_i$ , and to  $r_j$ . This gives us a hint as to the data dependencies. In order to update  $a_{ij}$ , one needs a copy of  $a_{ij}$ , and data from whichever processor owns  $c_i$  and whichever owns  $r_j$ . Since  $A$  is distributed across processors we will generally need communication to effect the subblock update step. Our job will be to analyze the associated costs, so that we can predict time to solution for given values of  $m$ ,  $n$ , and  $P$  on a particular architecture.

**Data Layout:** Assume that we have complete freedom in how the matrix elements are assigned to processors. (This may seem like an overly broad assumption given that moving the matrix to a particular layout involves an expensive all-to-all. In fact, as we will see, “data layout” really just relates to factorization order, i.e., the choice of pivots, which does not need to be  $a_{11}$ ,  $a_{22}$ , etc.)

Possible layouts include row partitioning. For example, if

$$A = \begin{bmatrix} \text{-----} r_1^T \text{-----} \\ \text{-----} r_2^T \text{-----} \\ \text{-----} \vdots \text{-----} \\ \text{-----} r_m^T \text{-----} \end{bmatrix}, \quad (3)$$

one could partition the row index set  $\mathcal{I} = [1 : m]$  into nonoverlapping subsets  $\mathcal{I}_0, \mathcal{I}_1, \dots, \mathcal{I}_{P-1}$ , and map any row  $i$  to processor (MPI rank)  $p$  if  $i \in \mathcal{I}_p$ . If we take the straightforward linear partition,  $\mathcal{I}_p = \{o, o+1, \dots, o+m/P-1\}$ , with offset  $o = 1 + (m/P)p$  for  $p = 0, \dots, P-1$ , then the first  $m/P$  pivots will be drawn from rows on rank 0. After that, they will be drawn from rank 1, and so forth. The net effect is that, one-by-one, processors will be idled, which will lead to significant load imbalance.

**Quick Quiz:** Under such a consecutive ordering/partitioning, how long would your “1 minute” calculation from the preceding quiz take to complete?

**Interleaved Layout:** The usual approach to avoiding load imbalance and minimizing communication overhead is to partition *both rows and columns* and to interleave these row/column distributions across the processors. An organized approach to doing so is to use a dual-index notation for the processor ranks. Assume that the ranks are labeled  $(p, q)$  for  $p = 0, \dots, P-1$  and  $q = 0, \dots, Q-1$  such that the total number of ranks is  $PQ$ . An example is illustrated below for the case where an  $8 \times 8$  matrix  $A$  is mapped to six ranks using  $P = 2$  and  $Q = 3$ .

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} & a_{18} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} & a_{28} \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & a_{36} & a_{37} & a_{38} \\ a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} & a_{47} & a_{48} \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & a_{57} & a_{58} \\ a_{61} & a_{62} & a_{63} & a_{64} & a_{65} & a_{66} & a_{67} & a_{68} \\ a_{71} & a_{72} & a_{73} & a_{74} & a_{75} & a_{76} & a_{77} & a_{78} \\ a_{81} & a_{82} & a_{83} & a_{84} & a_{85} & a_{86} & a_{87} & a_{88} \end{bmatrix}}_{\text{matrix entries}} \iff \underbrace{\begin{bmatrix} 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \end{bmatrix}}_{\text{processor mapping}} \quad (4)$$

Here, we assign  $a_{11}$  to rank  $(0,0)$ ,  $a_{12}$  to rank  $(0,1)$ , and so forth. The pattern continues until column  $Q+1$ , where  $a_{14}$  is also assigned to  $(0,0)$ . The row-wise distribution follows a similar pattern, with rows  $a_{i,:}$  assigned to ranks  $(0,*)$  when  $i$  is odd, or to ranks  $(1,*)$  when  $i$  is even. If we ignore pivoting, this partitioning will ensure that each rank will be active as the LU factorization proceeds all the way down to the final  $3 \times 3$  subblock, so load balance is effectively ensured for the majority of the computation.

Note that, in practice, we need a small look-up table or formula such that any processor can find the “true” MPI rank,  $p'$ , given a  $P \times Q$  processor distribution.

## Parallel Factorization

Here, we illustrate a typical factorization step for the  $(P, Q) = (2, 3)$  layout. We assume that rows/columns 1 and 2 have already been updated and that  $a_{kk}$  is the current pivot with  $k=3$ . As is usual, row  $k$  does not get modified, but column  $k$  is to be zeroed out for entries below the pivot (i.e., for  $i > k$ ). We use those locations to store the multiplier column,  $a_{ik}/a_{kk}$ . In expression (5) we highlight the pivot in red, the pivot row in pink, and the multiplier column in blue. The matrix on the right shows the (dual-indexed) processors that hold each entry. The pivot and the pivot row must be broadcast down to all active ranks below. Notice that,  $\forall q^* \in \{0, \dots, Q-1\}$ , any rank  $(0, q^*)$  will broadcast the pivot row to all ranks  $(p, q^*)$ ,  $\forall p \in \{0, \dots, P-1\}$ . This broadcast will require at most  $\log_2 P$  messages. Notice also that we can send the pivot in the same message batch for the particular set of ranks  $(p, 2)$ ,  $p \in \{0, \dots, P-1\}$ .

Once the multiplier column is formed, it must similarly be broadcast row-wise, with an overhead of  $\log_2 Q$  messages.

$$\left[ \begin{array}{cccccccc} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} & a_{18} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} & a_{28} \\ a_{31} & a_{32} & \mathbf{a_{33}} & a_{34} & a_{35} & a_{36} & a_{37} & a_{38} \\ a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} & a_{47} & a_{48} \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & a_{57} & a_{58} \\ a_{61} & a_{62} & a_{63} & a_{64} & a_{65} & a_{66} & a_{67} & a_{68} \\ a_{71} & a_{72} & a_{73} & a_{74} & a_{75} & a_{76} & a_{77} & a_{78} \\ a_{81} & a_{82} & a_{83} & a_{84} & a_{85} & a_{86} & a_{87} & a_{88} \end{array} \right] \iff \left[ \begin{array}{cccccccc} 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & \mathbf{0,2} & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \\ 0,0 & 0,1 & 0,2 & 0,0 & 0,1 & 0,2 & 0,0 & 0,1 \\ 1,0 & 1,1 & 1,2 & 1,0 & 1,1 & 1,2 & 1,0 & 1,1 \end{array} \right] \quad (5)$$

Below, we have the processor-local view for process  $(0,2)$ . We present both the *global numbering* (left) and *local numbering* (right). It is the local numbering that matters from a coding standpoint since our code needs to index only into the local arrays when using the distributed (private) memory model. Note that local numbering offers a several significant advantages over global addressing: (i) the parallel code looks more like serial code—arrays are indexed according to local ranges just as with serial code; (ii) array ranges are small and can generally be indexed with 32-bit integers (i.e., up to a billion  $\sim 2^{30}$ ), rather than 64-bit, which would be required for most global index ranges. (iii) there is a clear distinction between local (fast) memory accesses and global (slow) ones, which serves to clarify the programming approach for realizing efficient, communication minimal, performance. For clarity, one can think of  $a_{11}$  on the right (of which there are  $P \times Q$  copies) as having an implicit superscript,  $a_{11}^{p,q}$ . This implied processor-index is sufficient to disambiguate memory references in the the private-memory model of distributed memory computing. In all of our assignments, we work exclusively with the **local index numbering**.

$$A_{0,2} = \underbrace{\left[ \begin{array}{cc} a_{13} & a_{16} \\ \mathbf{a_{33}} & a_{36} \\ a_{53} & a_{56} \\ a_{73} & a_{76} \end{array} \right]}_{\text{Global Numbering}} \quad \underbrace{\left[ \begin{array}{cc} a_{11} & a_{12} \\ \mathbf{a_{21}} & \mathbf{a_{22}} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \end{array} \right]}_{\text{Local Numbering}} \quad (6)$$

In expression (6), we have grayed out the part of the array that has already been factored (i.e., the rows of  $A$  that are now part of  $U$ ). The pivot, which is owned solely by rank  $(0,2)$ , is in red, and the portions of the pivot row and multiplier column are defined respectively in pink and blue. The white subblock is the part that will be updated for this particular step where the (global) pivot is  $a_{33}$ .

Obviously, rank (0,2) does not require communication to proceed with this local update, but it does need to broadcast the pivot and pivot row (pink) downward to rank (1,2) and it also needs to broadcast the multiplier column (once computed) rightward to ranks (0,0) and (0,1), as is evident from inspection of (5). In a similar fashion, we see that ranks (0,  $q$ ), will need to broadcast their pivot row entries to ranks ( $p, q$ ), for  $p \neq 0$  and  $q = 0 : Q - 1$ . Once the pivot is broadcast, the multiplier column can be computed on each rank, ( $p, 2$ ), and broadcast row-wise to corresponding ranks ( $p, q$ ),  $q \neq 2$ .

Notice that, for each rank ( $p, q$ ), there are only two types of broadcasts: column-wise and row-wise. Each of these broadcasts involves, respectively,  $P$  or  $Q$  ranks and can be completed in  $\log_2 P$  or  $\log_2 Q$  messages involving a single-word broadcast. Moreover, row broadcasts will only involve process index sets of the form ( $p, :$ ) and never require data transmission to ( $p + i, :$ ),  $i \neq 0$ . Similarly, each column broadcast only require interactions within sets of the form ( $:, q$ ).

Notice that the *root* (i.e., source) of the broadcast changes as successive pivots  $a_{kk}$  are selected during the factorization process. One can either implement the broadcast as an all-reduce or develop a root-specific broadcast. The former requires  $\lceil 2 \log_2 P \rceil$  communication steps while the latter requires only  $\lceil \log_2 P \rceil$  steps.<sup>1</sup>

---

<sup>1</sup>A notable exception was the set of IBM BG/L-BG/Q architectures, which supported hardware all-reduce in which vector reductions went through the MPI software stack just once per invocation, rather than  $2 \log_2 P$  times. On these machines, vector-reduction costs were about  $4\times$  that of message-passing latency, independent of  $P$ , even for  $P > 10^6$ .